SparkIIT AI Internship

Project 2 Report | Machine Learning, Predictive Analytics, Classification, Data Analysis

AI-Based Predictive Decision Support System

Kaushikee Karmakar

Registered Email: kaushikeekarmakar@gmail.com

BCA, Asansol Engineering College

Program: SparkIIT AI Training and Internship (October 2026)

Deliverables: Python notebook (.ipynb) with outputs, web interface code and this report

GitHub repository: https://github.com/KK-Dev0/Predictive-Decision-Support-AI

Live app: https://predictive-decision-support-ai.onrender.com

---

Contents

No.	Section
1	Abstract
2	Introduction
3	Objectives
4	Literature Review / Existing System
5	Methodology
6	Code
7	Output Snapshots
8	Results & Analysis
9	Conclusion

1. Abstract

Lenders must decide whether to approve a loan using imperfect information about whether the borrower will repay. This project builds an AI-based predictive decision support system that estimates the probability that a borrower will become seriously delinquent (90 or more days past due) within two years. It uses the Kaggle *Give Me Some Credit* data, 149,999 real borrowers of whom 6.68% defaulted. After cleaning, feature engineering and a leak-free pipeline, three models (Logistic Regression, Random Forest and Hist Gradient Boosting) were compared with 5-fold cross-validation using ROC-AUC, PR-AUC and Brier score, because plain accuracy is misleading when only 6.7% of cases are positive. A tuned Random Forest was selected and reached a test ROC-AUC of 0.868, a PR-AUC of 0.396 (against 0.067 for a model with no skill) and a Brier score of 0.049. A decision threshold chosen under an assumed 5:1 cost of errors declines 11% of applicants, catches 58% of eventual defaulters and lowers the default rate among approved borrowers to 3.1% from 6.7%, a 37% lower expected cost than the best policy without a model. The system explains each estimate, was audited for differences across age groups, and provides an interactive prediction interface in the notebook and as a web application.

2. Introduction

Credit risk assessment is a classic decision-support problem. A lender gains from every good loan and loses much more from a loan that is not repaid, so even a modest improvement in identifying risky borrowers has real value. At the same time, such decisions affect people's access to credit, which makes explanations and fairness as important as accuracy.

Three features of this problem shape the project. First, the data is heavily imbalanced: only about one borrower in fifteen defaults, so a model that approves everybody is already 93% accurate and completely useless. Second, the output that matters is a probability that can be turned into a decision using the costs of the two kinds of mistake, not just a yes or no label. Third, a decision-support tool should show why it produced a score and should be checked for unequal treatment of groups.

This project therefore treats the task end to end: data cleaning, feature engineering, model comparison with suitable metrics, a cost-based decision rule, explanation, a fairness audit and an interactive interface. The system is intended to support a human decision maker and not to make decisions on its own.

3. Objectives

- Collect and preprocess a real credit dataset, handling missing values, impossible values and special codes without leaking test data.

- Perform feature engineering and feature selection.
- Train multiple machine learning models and compare their performance with metrics suited to imbalanced data.
- Generate predictions with confidence scores (probabilities), check their calibration and convert them into approve / review / decline recommendations using a cost-based threshold.
- Explain individual predictions and overall model behavior.
- Audit the decisions for differences across age groups.
- Develop an interactive prediction interface for the final model.
- Deliver a fully executed notebook with visible outputs and a report.

4. Literature Review / Existing System

Judgmental underwriting. Traditionally, loan officers decide using experience and a review of the application. This is flexible but slow, inconsistent between people, and hard to audit.

Statistical scorecards. Credit scoring in industry has long relied on logistic-regression scorecards built from credit-bureau variables such as payment history, utilization and debt. They are valued because they are simple, stable and explainable, and because each factor's contribution can be stated.

Machine learning models. Tree ensembles such as Random Forests and gradient boosting can capture non-linear effects and interactions and often rank borrowers better than a single linear model, but they are harder to interpret, which creates a need for explanation methods such as permutation importance and per-decision sensitivity checks.

Explainability and fairness. Lending is a regulated area in which decisions generally have to be explainable and must not discriminate unfairly. Models should therefore be audited for differences in outcomes across groups, keeping in mind that removing a sensitive column does not remove its influence if other variables act as proxies.

Benchmark. The Kaggle competition *Give Me Some Credit* is a well-known public benchmark for predicting serious delinquency and is used here.

Limitations of common practice that this project addresses. Many student and demonstration projects report accuracy on imbalanced data, ignore the cost of different errors, give a score with no explanation, and never check group-level effects. This project instead uses ranking and calibration metrics, derives a decision rule from explicit costs, explains the estimate, and includes a fairness audit.

References. Kaggle, Give Me Some Credit, https://www.kaggle.com/c/GiveMeSomeCredit. scikit-learn documentation, https://scikit-learn.org. Flask documentation, https://flask.palletsprojects.com.

5. Methodology

5.1 Dataset

The file *cs-training.csv* of the Kaggle competition contains 150,000 borrowers with ten inputs (credit utilization, age, three past-due counts, debt ratio, monthly income, number of open credit lines, real estate loans and number of dependents) and the target *SeriousDlqin2yrs*, which is 1 if the borrower became 90 or more days past due within two years. After removing one row with an impossible age, 149,999 borrowers remained, of whom 6.68% are positive.

5.2 Data quality and cleaning

Monthly income and number of dependents contain missing values. The three past-due columns contain error codes (96 and 98), which were treated as missing. Utilization, debt ratio and income contain extreme outliers, which were capped at percentiles of the training data (99th for utilization and debt ratio, 99.5th for income).

5.3 Split and feature engineering

The data was split 80/20 with stratification into 119,999 training and 30,000 test borrowers before any statistic was computed. Five features were added: total past-due events, an ever-late flag, log of monthly income, log of debt ratio and income per dependent.

5.4 Pipeline

A single scikit-learn pipeline performs median imputation with missing-value indicators, standardization, feature selection with an ANOVA F-test (the number of features is tuned), and the model. Keeping every step inside the pipeline means cross-validation never leaks information from validation folds. No class weighting was used, so predicted probabilities stay meaningful.

5.5 Models and comparison

Logistic Regression, Random Forest and Hist Gradient Boosting were compared with stratified 5-fold cross-validation using ROC-AUC (ranking quality), PR-AUC (precision-recall trade-off, informative under imbalance) and Brier score (quality of the probabilities). Accuracy was deliberately not used.

5.6 Tuning

The model with the best cross-validated ROC-AUC was tuned with 3-fold GridSearchCV over its main hyperparameters and the number of selected features.

5.7 Calibration and cost-based decision rule

Calibration was checked with a calibration curve and the Brier score. The decision threshold was chosen on out-of-fold training predictions by minimizing an assumed cost in which approving a borrower who later defaults costs 5 times as much as declining a good borrower, and was then checked on the untouched test set. Recommendations are: decline at or above the threshold, manual review from half the threshold, otherwise approve.

5.8 Explainability

Permutation importance was computed on the original inputs using a 10,000-row sample of the test set (features rebuilt after each shuffle). For individual borrowers, each input is replaced by a typical value (the median) to measure how much the estimate changes.

5.9 Fairness audit

Age is the only sensitive attribute in the data. On the test set, decline rates, false-decline rates and default-catch rates were compared across three age groups (under 30, 30 to 59, 60 and over), with age in the model and with age removed.

5.10 Prediction interface

The interface exists in two forms. In the notebook, the function *assess()* takes a borrower's details and returns the risk estimate, the recommendation and the main drivers. As a web application, a Flask backend loads the saved model and exposes a prediction endpoint that a page built with HTML, CSS and JavaScript calls; the page contains an input form, a risk scale with the recommendation, a breakdown of the drivers, and performance and fairness sections.

5.11 Tools

Python 3, scikit-learn, pandas, NumPy, Matplotlib, joblib, Flask, HTML/CSS/JavaScript and Google Colab.

6. Code

The complete notebook code is listed below, cell by cell, as executed (very long lines are wrapped in this document). It is also provided as the notebook *Project2_Predictive_Decision_Support.ipynb* with all outputs visible. The Flask backend of the web interface follows the notebook code.

Notebook cell 1

```
import os
import zipfile
import glob
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import joblib
from IPython.display import display
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate, cross_val_predict,
    GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.base import clone
from sklearn.metrics import (roc_auc_score, average_precision_score, brier_score_loss, roc_curve,
                             precision_recall_curve, precision_score, recall_score, f1_score)
from sklearn.calibration import calibration_curve

warnings.filterwarnings("ignore")
plt.rcParams.update({"figure.dpi": 110, "axes.grid": True, "grid.alpha": 0.3})
RANDOM_STATE = 42
TARGET = "SeriousDlqin2yrs"
print("Libraries loaded")
```

Notebook cell 2

```
if not os.path.exists("cs-training.csv"):
    try:
        from google.colab import files
        print("Choose cs-training.csv (or the Kaggle zip) in the file picker...")
        uploaded = files.upload()
    except ImportError:
        raise FileNotFoundError("Put cs-training.csv in the same folder as this notebook.")
    for name in uploaded:
        if name.lower().endswith(".zip"):
            with zipfile.ZipFile(name) as z:
                member = [m for m in z.namelist() if m.lower().endswith("cs-training.csv")][0]
                z.extract(member, ".")
                os.replace(member, "cs-training.csv")
        elif name != "cs-training.csv":
            os.replace(name, "cs-training.csv")
raw = pd.read_csv("cs-training.csv")
print("Loaded:", raw.shape)
```

Notebook cell 3

```
df = raw.loc[:, ~raw.columns.str.startswith("Unnamed")].copy()
print("Rows, columns:", df.shape)
print(f"Serious delinquency rate: {df[TARGET].mean():.4f}  ({int(df[TARGET].sum())} of {len(df)} borrowers)")
print("\nMissing values per column:")
print(df.isna().sum()[df.isna().sum() > 0])
print("\nData problems found:")
print("  age below 18:", int((df["age"] < 18).sum()))
for c in ["NumberOfTime30-59DaysPastDueNotWorse", "NumberOfTime60-89DaysPastDueNotWorse",
    "NumberOfTimes90DaysLate"]:
    print(f"  {c} >= 90 (special codes):", int((df[c] >= 90).sum()))
print("  utilization above 2 (over 200% of limits):", int((df["RevolvingUtilizationOfUnsecuredLines"] > 2).sum()))
before = len(df)
df = df[df["age"] >= 18].reset_index(drop=True)
print(f"\nDropped {before - len(df)} row(s) with impossible age. Rows left: {len(df)}")
df.describe().T.round(2)
```

Notebook cell 4

```
fig, ax = plt.subplots(1, 3, figsize=(14, 4))
bands = pd.cut(df["age"], [17, 29, 44, 59, 120], labels=["18-29", "30-44", "45-59", "60+"])
df.groupby(bands)[TARGET].mean().mul(100).plot(kind="bar", ax=ax[0], color="#4F46E5", rot=0)
ax[0].set_title("Delinquency rate by age band (%)")
df.groupby(df["MonthlyIncome"].isna().map({True: "income missing", False: "income
    reported"}))[TARGET].mean().mul(100).plot(kind="bar", ax=ax[1], color="#DB2777", rot=0)
ax[1].set_title("Delinquency rate: missing income (%)")
pdue = df[["NumberOfTime30-59DaysPastDueNotWorse", "NumberOfTime60-89DaysPastDueNotWorse",
    "NumberOfTimes90DaysLate"]].where(lambda x: x < 90).sum(axis=1)
df.groupby(pd.cut(pdue, [-1, 0, 1, 2, 100], labels=["0", "1", "2", "3+"]))[TARGET].mean().mul(100).plot(kind="bar",
    ax=ax[2], color="#0F766E", rot=0)
ax[2].set_title("Delinquency rate by past-due events (%)")
plt.tight_layout()
plt.savefig("fig_eda.png", dpi=150)
plt.show()
```

Notebook cell 5

```
train_df, test_df = train_test_split(df, test_size=0.2, random_state=RANDOM_STATE, stratify=df[TARGET])
y_train, y_test = train_df[TARGET], test_df[TARGET]
X_train_raw, X_test_raw = train_df.drop(columns=TARGET), test_df.drop(columns=TARGET)
caps = {"util": float(X_train_raw["RevolvingUtilizationOfUnsecuredLines"].quantile(0.99)),
        "debt": float(X_train_raw["DebtRatio"].quantile(0.99)),
        "income": float(X_train_raw["MonthlyIncome"].quantile(0.995))}
print("Caps from training data:", {k: round(v, 2) for k, v in caps.items()})

PAST_DUE = ["NumberOfTime30-59DaysPastDueNotWorse", "NumberOfTime60-89DaysPastDueNotWorse",
    "NumberOfTimes90DaysLate"]


def add_features(d, caps):
    d = d.copy()
    for c in PAST_DUE:
        d[c] = d[c].where(d[c] < 90)   # codes 96 and 98 are data errors, treated as missing
    d["RevolvingUtilizationOfUnsecuredLines"] = d["RevolvingUtilizationOfUnsecuredLines"].clip(upper=caps["util"])
    d["DebtRatio"] = d["DebtRatio"].clip(upper=caps["debt"])
    d["MonthlyIncome"] = d["MonthlyIncome"].clip(upper=caps["income"])
    d["total_past_due"] = d[PAST_DUE].sum(axis=1, min_count=1)
    d["ever_late"] = (d["total_past_due"].fillna(0) > 0).astype(int)
    d["log_monthly_income"] = np.log1p(d["MonthlyIncome"])
    d["log_debt_ratio"] = np.log1p(d["DebtRatio"])
    d["income_per_dependent"] = d["MonthlyIncome"] / (d["NumberOfDependents"] + 1)
    return d


X_train = add_features(X_train_raw, caps)
X_test = add_features(X_test_raw, caps)
print("Train:", X_train.shape, "| Test:", X_test.shape)
print("Delinquency rate  train: %.4f  test: %.4f" % (y_train.mean(), y_test.mean()))
print("Engineered:", [c for c in X_train.columns if c not in X_train_raw.columns])
```

Notebook cell 6

```
def build(model, k="all"):
    return Pipeline([
        ("imp", SimpleImputer(strategy="median", add_indicator=True)),
        ("sc", StandardScaler()),
        ("select", SelectKBest(f_classif, k=k)),
        ("model", model),
    ])


models = {
    "Logistic Regression": LogisticRegression(max_iter=2000),
    "Random Forest": RandomForestClassifier(n_estimators=150, max_depth=10, min_samples_leaf=20, n_jobs=-1,
        random_state=RANDOM_STATE),
    "Hist Gradient Boosting": HistGradientBoostingClassifier(max_iter=300, random_state=RANDOM_STATE),
}
print("Models:", list(models))
```

Notebook cell 7

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = {"ROC-AUC": "roc_auc", "PR-AUC": "average_precision", "Brier": "neg_brier_score"}
rows = []
for name, m in models.items():
    res = cross_validate(build(m), X_train, y_train, cv=cv, scoring=scoring, n_jobs=1)
    row = {"Model": name}
    for label in scoring:
        vals = res["test_" + label] * (-1 if label == "Brier" else 1)
        row[label] = f"{vals.mean():.4f} +/- {vals.std():.4f}"
        row[label + "_mean"] = vals.mean()
    rows.append(row)
cv_df = pd.DataFrame(rows).set_index("Model")
cv_df[list(scoring)]
```

Notebook cell 8

```
ax = cv_df[["ROC-AUC_mean", "PR-AUC_mean"]].rename(columns={"ROC-AUC_mean": "ROC-AUC", "PR-AUC_mean":
    "PR-AUC"}).plot(
    kind="bar", figsize=(8, 4), rot=10, color=["#4F46E5", "#DB2777"])
ax.set_title("Cross-validated model comparison")
plt.tight_layout()
plt.savefig("fig_cv_comparison.png", dpi=150)
plt.show()
```

Notebook cell 9

```
grids = {
    "Logistic Regression": {"select__k": [8, "all"], "model__C": [0.1, 1, 10]},
    "Random Forest": {"select__k": [8, "all"], "model__max_depth": [8, 12], "model__min_samples_leaf": [20, 50]},
    "Hist Gradient Boosting": {"select__k": [8, "all"], "model__learning_rate": [0.05, 0.1],
                               "model__max_leaf_nodes": [15, 31], "model__l2_regularization": [0.0, 1.0]},
}
best_name = cv_df["ROC-AUC_mean"].idxmax()
base_auc = cv_df.loc[best_name, "ROC-AUC_mean"]
cv3 = StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE)
grid = GridSearchCV(build(models[best_name]), grids[best_name], cv=cv3, scoring="roc_auc", n_jobs=1)
grid.fit(X_train, y_train)
best_pipe = grid.best_estimator_
print("Selected model:", best_name)
print("Best params:", grid.best_params_)
print(f"CV ROC-AUC before tuning (5-fold): {base_auc:.4f} | tuned (3-fold): {grid.best_score_:.4f}")
print("(the two numbers use different folds, so a tiny difference means tuning changed little)")
try:
```

```
    kept = best_pipe[:-1].get_feature_names_out()
    print(len(kept), "features kept after selection")
except Exception as e:
    print("Could not list kept features:", e)
```

Notebook cell 10

```
proba = best_pipe.predict_proba(X_test)[:, 1]
test_metrics = {
    "ROC-AUC": roc_auc_score(y_test, proba),
    "PR-AUC": average_precision_score(y_test, proba),
    "Brier": brier_score_loss(y_test, proba),
}
base_rate = float(y_test.mean())
print(f"Test rows: {len(y_test)} | delinquency rate: {base_rate:.4f}")
for k, v in test_metrics.items():
    print(f"{k:8s}: {v:.4f}")
print(f"PR-AUC of a model with no skill equals the base rate: {base_rate:.4f}")
print(f"Accuracy of 'approve everyone': {1 - base_rate:.4f} (this is why accuracy is not used)")

fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
fpr, tpr, _ = roc_curve(y_test, proba)
ax[0].plot(fpr, tpr, color="#4F46E5", lw=2, label=f"ROC-AUC = {test_metrics['ROC-AUC']:.3f}")
ax[0].plot([0, 1], [0, 1], "--", color="grey")
ax[0].set_xlabel("False positive rate"); ax[0].set_ylabel("True positive rate"); ax[0].set_title("ROC curve");
    ax[0].legend()
pr, rc, _ = precision_recall_curve(y_test, proba)
ax[1].plot(rc, pr, color="#DB2777", lw=2, label=f"PR-AUC = {test_metrics['PR-AUC']:.3f}")
ax[1].axhline(base_rate, color="grey", ls="--", label="no-skill baseline")
ax[1].set_xlabel("Recall"); ax[1].set_ylabel("Precision"); ax[1].set_title("Precision-recall curve"); ax[1].legend()
plt.tight_layout()
plt.savefig("fig_test_eval.png", dpi=150)
plt.show()
```

Notebook cell 11

```
COST_BAD_APPROVED, COST_GOOD_DECLINED = 5, 1


def cost_at(t, yt, p):
    decline = p >= t
    return COST_BAD_APPROVED * np.sum(~decline & (yt == 1)) + COST_GOOD_DECLINED * np.sum(decline & (yt == 0))


oof = cross_val_predict(best_pipe, X_train, y_train, cv=cv, method="predict_proba")[:, 1]
ths = np.linspace(0.01, 0.60, 119)
costs = np.array([cost_at(t, y_train.values, oof) for t in ths]) / len(y_train)
t_star = float(ths[int(np.argmin(costs))])
t_review = 0.5 * t_star
n = len(y_test)
cost_all = COST_BAD_APPROVED * y_test.mean()
cost_none = COST_GOOD_DECLINED * (1 - y_test.mean())
cost_model = cost_at(t_star, y_test.values, proba) / n
print(f"Decline threshold (cost-optimal, from cross-validation): {t_star:.3f}")
print(f"Average cost per applicant on the test set:")
print(f"   approve everyone : {cost_all:.4f}")
print(f"   decline everyone : {cost_none:.4f}")
print(f"   this model       : {cost_model:.4f}  ({(1 - cost_model / min(cost_all, cost_none)) * 100:.1f}% below the
    best no-model policy)")

decline = proba >= t_star
policy = {
    "Decline rate": float(decline.mean()),
    "Default rate among approved": float(y_test[~decline].mean()),
    "Default rate among declined": float(y_test[decline].mean()),
    "Share of all defaulters caught (recall)": float(recall_score(y_test, decline)),
    "Precision of declines": float(precision_score(y_test, decline)),
    "F1 of declines": float(f1_score(y_test, decline)),
}
print("\nDecision at the threshold, on the test set:")
for k, v in policy.items():
    print(f"   {k}: {v:.3f}")
print(f"   (overall default rate: {base_rate:.3f})")

frac_pos, mean_pred = calibration_curve(y_test, proba, n_bins=10, strategy="quantile")
fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
ax[0].plot(mean_pred, frac_pos, "o-", color="#4F46E5", label="Model")
lim = max(mean_pred.max(), frac_pos.max()) * 1.05
ax[0].plot([0, lim], [0, lim], "--", color="grey", label="Perfect calibration")
ax[0].set_xlabel("Predicted probability of delinquency"); ax[0].set_ylabel("Actual delinquency rate")
ax[0].set_title(f"Calibration (Brier {test_metrics['Brier']:.4f})"); ax[0].legend()
ax[1].plot(ths, costs, color="#DB2777")
ax[1].axvline(t_star, color="grey", ls="--")
ax[1].set_xlabel("Decline threshold"); ax[1].set_ylabel("Average cost per applicant"); ax[1].set_title("Cost vs
    decline threshold")
plt.tight_layout()
plt.savefig("fig_calibration_cost.png", dpi=150)
plt.show()
```

Notebook cell 12

```
def raw_importance(pipe, Xraw, y, caps, n_repeats=5, n=10000, seed=RANDOM_STATE):
    rng = np.random.default_rng(seed)
    idx = rng.choice(len(Xraw), size=min(n, len(Xraw)), replace=False)
    Xs, ys = Xraw.iloc[idx].reset_index(drop=True), y.iloc[idx].reset_index(drop=True)
    base = roc_auc_score(ys, pipe.predict_proba(add_features(Xs, caps))[:, 1])
```

```
    out = {}
    for c in Xs.columns:
        drops = []
        for _ in range(n_repeats):
            Xp = Xs.copy()
            Xp[c] = rng.permutation(Xp[c].values)
            drops.append(base - roc_auc_score(ys, pipe.predict_proba(add_features(Xp, caps))[:, 1]))
        out[c] = float(np.mean(drops))
    return pd.Series(out).sort_values(ascending=False)


imp = raw_importance(best_pipe, X_test_raw, y_test, caps)
top = imp.head(10)[::-1]
plt.figure(figsize=(8, 4.8))
plt.barh(top.index, top.values, color="#4F46E5")
plt.xlabel("Drop in ROC-AUC when shuffled")
plt.title("Most influential inputs (permutation importance)")
plt.tight_layout()
plt.savefig("fig_importance.png", dpi=150)
plt.show()
imp.round(4)
```

Notebook cell 13

```
age_group = np.select([X_test_raw["age"] < 30, X_test_raw["age"] < 60], ["under 30", "30 to 59"], default="60 and
    over")
y_arr = y_test.values


def group_table(p):
    d = p >= t_star
    out = []
    for g in ["under 30", "30 to 59", "60 and over"]:
        m = age_group == g
        out.append({"Age group": g, "n": int(m.sum()), "Actual default rate": y_arr[m].mean(),
                    "Mean predicted risk": p[m].mean(), "Declined": d[m].mean(),
                    "Declined among non-defaulters": d[m & (y_arr == 0)].mean(),
                    "Caught among defaulters": d[m & (y_arr == 1)].mean()})
    return pd.DataFrame(out).set_index("Age group").round(3)


fair_with = group_table(proba)
noage_pipe = clone(best_pipe).fit(X_train.drop(columns="age"), y_train)
proba_noage = noage_pipe.predict_proba(X_test.drop(columns="age"))[:, 1]
fair_without = group_table(proba_noage)
auc_with, auc_without = roc_auc_score(y_test, proba), roc_auc_score(y_test, proba_noage)
print(f"Test ROC-AUC with age: {auc_with:.4f} | without age: {auc_without:.4f}\n")
print("WITH age in the model:")
display(fair_with)
print("WITHOUT age in the model:")
display(fair_without)
```

Notebook cell 14

```
ui_steps = {"RevolvingUtilizationOfUnsecuredLines": 0.01, "DebtRatio": 0.05, "MonthlyIncome": 100.0}
ui_stats = {}
for c in X_train_raw.columns:
    hi = float(np.ceil(X_train_raw[c].quantile(0.995)))
    med = float(X_train_raw[c].median())
    ui_stats[c] = {"min": 18.0 if c == "age" else 0.0, "max": max(hi, med + 1.0), "median": med, "step":
        ui_steps.get(c, 1.0)}
meta = {
    "pipeline": best_pipe,
    "model_name": best_name,
    "caps": caps,
    "raw_cols": X_train_raw.columns.tolist(),
    "ui_stats": ui_stats,
    "t_star": t_star,
    "t_review": t_review,
    "base_rate": float(y_train.mean()),
    "cv_table": cv_df[list(scoring)].reset_index().to_dict("records"),
    "test_metrics": {k: float(v) for k, v in test_metrics.items()},
    "policy": policy,
    "importance": {k: float(v) for k, v in imp.head(10).round(4).items()},
    "fair_with": fair_with.reset_index().to_dict("records"),
    "fair_without": fair_without.reset_index().to_dict("records"),
    "auc_with_age": float(auc_with),
    "auc_without_age": float(auc_without),
    "n_train": int(len(X_train)),
    "n_test": int(len(X_test)),
}
joblib.dump(meta, "credit_model.joblib")
import sklearn, sys
print("Saved credit_model.joblib | size: %.1f MB" % (os.path.getsize("credit_model.joblib") / 1e6))
print("sklearn:", sklearn.__version__, "| python:", sys.version.split()[0])
try:
    from google.colab import files
    files.download("credit_model.joblib")
except ImportError:
    print("Not running in Colab, file saved in the current folder.")
```

Notebook cell 15

```
lines = ["=== PROJECT 2 SUMMARY (copy everything from here to the end and send it) ==="]
lines.append(f"Dataset: Give Me Some Credit, {len(df)} rows after cleaning, delinquency rate
    {df[TARGET].mean():.4f}")
```

```
lines.append(f"Train/test: {len(X_train)}/{len(X_test)}")
lines.append("CV results (5-fold, train set):\n" + cv_df[list(scoring)].to_string())
lines.append(f"Selected: {best_name} | params: {grid.best_params_} | CV AUC 5-fold {base_auc:.4f}, tuned 3-fold
    {grid.best_score_:.4f}")
lines.append("Test metrics: " + str({k: round(float(v), 4) for k, v in test_metrics.items()}) + f" | base rate
    {base_rate:.4f}")
lines.append(f"Decline threshold {t_star:.3f} (review from {t_review:.3f}); cost per applicant: approve-all
    {cost_all:.4f}, decline-all {cost_none:.4f}, model {cost_model:.4f}")
lines.append("Policy on test: " + str({k: round(v, 3) for k, v in policy.items()}))
lines.append("Importance (drop in ROC-AUC):\n" + imp.head(10).round(4).to_string())
lines.append(f"Age fairness: test AUC with age {auc_with:.4f}, without {auc_without:.4f}")
lines.append("WITH age:\n" + fair_with.to_string())
lines.append("WITHOUT age:\n" + fair_without.to_string())
print("\n".join(lines))

with zipfile.ZipFile("project2_figures.zip", "w") as z:
    for f in glob.glob("fig_*.png"):
        z.write(f)
try:
    from google.colab import files
    files.download("project2_figures.zip")
except ImportError:
    pass
```

Notebook cell 16

```
import os
import numpy as np
import pandas as pd
import joblib

if not os.path.exists("credit_model.joblib"):
    from google.colab import files
    print("Upload credit_model.joblib (saved earlier by this notebook)")
    files.upload()
ui = joblib.load("credit_model.joblib")

PAST_DUE = ["NumberOfTime30-59DaysPastDueNotWorse", "NumberOfTime60-89DaysPastDueNotWorse",
    "NumberOfTimes90DaysLate"]


def add_features(d, caps):
    d = d.copy()
    for c in PAST_DUE:
        d[c] = d[c].where(d[c] < 90)   # codes 96 and 98 are data errors, treated as missing
    d["RevolvingUtilizationOfUnsecuredLines"] = d["RevolvingUtilizationOfUnsecuredLines"].clip(upper=caps["util"])
    d["DebtRatio"] = d["DebtRatio"].clip(upper=caps["debt"])
    d["MonthlyIncome"] = d["MonthlyIncome"].clip(upper=caps["income"])
    d["total_past_due"] = d[PAST_DUE].sum(axis=1, min_count=1)
    d["ever_late"] = (d["total_past_due"].fillna(0) > 0).astype(int)
    d["log_monthly_income"] = np.log1p(d["MonthlyIncome"])
    d["log_debt_ratio"] = np.log1p(d["DebtRatio"])
    d["income_per_dependent"] = d["MonthlyIncome"] / (d["NumberOfDependents"] + 1)
    return d


def assess(borrower):
    # Return the risk estimate and recommendation for one borrower, and print the main drivers.
    row = pd.DataFrame([borrower])[ui["raw_cols"]]
    frames = [row]
    for c in ui["raw_cols"]:
        alt = row.copy()
        alt[c] = ui["ui_stats"][c]["median"]
        frames.append(alt)
    probs = ui["pipeline"].predict_proba(add_features(pd.concat(frames, ignore_index=True), ui["caps"]))[:, 1]
    p = float(probs[0])
    band = "DECLINE" if p >= ui["t_star"] else "MANUAL REVIEW" if p >= ui["t_review"] else "APPROVE"
    effects = sorted(((c, p - probs[i + 1]) for i, c in enumerate(ui["raw_cols"])), key=lambda kv: -abs(kv[1]))
    print(f"Estimated probability of serious delinquency: {p * 100:.1f}%  ->  {band}")
    print(f"   (average borrower {ui['base_rate'] * 100:.1f}% | review from {ui['t_review'] * 100:.1f}% | decline
        from {ui['t_star'] * 100:.1f}%)")
    for c, e in effects[:3]:
        if abs(e) > 0.002:
            print(f"   {'raises' if e > 0 else 'lowers'} risk: {c} ({e * 100:+.1f} points)")
    return p, band


examples = {
    "Steady repayer": {"RevolvingUtilizationOfUnsecuredLines": 0.12, "age": 52,
        "NumberOfTime30-59DaysPastDueNotWorse": 0,
                       "DebtRatio": 0.25, "MonthlyIncome": 6500, "NumberOfOpenCreditLinesAndLoans": 9,
                           "NumberOfTimes90DaysLate": 0,
                       "NumberRealEstateLoansOrLines": 1, "NumberOfTime60-89DaysPastDueNotWorse": 0,
                           "NumberOfDependents": 2},
    "Heavy credit use": {"RevolvingUtilizationOfUnsecuredLines": 0.85, "age": 41,
        "NumberOfTime30-59DaysPastDueNotWorse": 1,
                         "DebtRatio": 0.55, "MonthlyIncome": 4200, "NumberOfOpenCreditLinesAndLoans": 12,
                             "NumberOfTimes90DaysLate": 0,
                         "NumberRealEstateLoansOrLines": 2, "NumberOfTime60-89DaysPastDueNotWorse": 0,
                             "NumberOfDependents": 3},
    "Recent late payments": {"RevolvingUtilizationOfUnsecuredLines": 0.95, "age": 29,
        "NumberOfTime30-59DaysPastDueNotWorse": 2,
                             "DebtRatio": 0.70, "MonthlyIncome": 3000, "NumberOfOpenCreditLinesAndLoans": 6,
                                 "NumberOfTimes90DaysLate": 1,
                             "NumberRealEstateLoansOrLines": 0, "NumberOfTime60-89DaysPastDueNotWorse": 1,
```

```
                                "NumberOfDependents": 1},
}
for name, borrower in examples.items():
    print(name)
    assess(borrower)
    print()
```

Web interface: Flask backend (app.py)

The page itself (index.html, style.css, app.js) is in the project repository.

app.py

```
import math
import os

import joblib
import numpy as np
import pandas as pd
from flask import Flask, jsonify, render_template, request

app = Flask(__name__)
META = joblib.load(os.path.join(os.path.dirname(os.path.abspath(__file__)), "credit_model.joblib"))
PIPE = META["pipeline"]

PAST_DUE = ["NumberOfTime30-59DaysPastDueNotWorse", "NumberOfTime60-89DaysPastDueNotWorse",
    "NumberOfTimes90DaysLate"]


def add_features(d, caps):
    d = d.copy()
    for c in PAST_DUE:
        d[c] = d[c].where(d[c] < 90)   # codes 96 and 98 are data errors, treated as missing
    d["RevolvingUtilizationOfUnsecuredLines"] = d["RevolvingUtilizationOfUnsecuredLines"].clip(upper=caps["util"])
    d["DebtRatio"] = d["DebtRatio"].clip(upper=caps["debt"])
    d["MonthlyIncome"] = d["MonthlyIncome"].clip(upper=caps["income"])
    d["total_past_due"] = d[PAST_DUE].sum(axis=1, min_count=1)
    d["ever_late"] = (d["total_past_due"].fillna(0) > 0).astype(int)
    d["log_monthly_income"] = np.log1p(d["MonthlyIncome"])
    d["log_debt_ratio"] = np.log1p(d["DebtRatio"])
    d["income_per_dependent"] = d["MonthlyIncome"] / (d["NumberOfDependents"] + 1)
    return d

# Cost figures printed by the training notebook (assumed 5:1 cost of a missed defaulter vs a wrongly declined good
    borrower).
REPORTED_COST = {"approve_all": 0.3342, "decline_all": 0.9332, "model": 0.2107}

# key: (label, hint, kind, suffix)
SPEC = {
    "age": ("Age", "Age is a sensitive attribute. See the fairness section.", "int", "years"),
    "MonthlyIncome": ("Monthly income", "Before tax. Tick 'Not known' if it was not reported.", "money", "per
        month"),
    "NumberOfDependents": ("Dependents", "People who rely on the borrower's income.", "int", ""),
    "RevolvingUtilizationOfUnsecuredLines": ("Credit utilization", "Card and credit-line balances as a share of
        total limits.", "percent", "%"),
    "DebtRatio": ("Debt ratio", "Monthly debt payments as a share of monthly income.", "percent", "%"),
    "NumberOfOpenCreditLinesAndLoans": ("Open credit lines and loans", "Loans plus credit cards currently open.",
        "int", ""),
    "NumberRealEstateLoansOrLines": ("Real estate loans", "Mortgages and home equity lines.", "int", ""),
    "NumberOfTime30-59DaysPastDueNotWorse": ("Payments 30 to 59 days late", "Times in the last two years.", "int",
        "times"),
    "NumberOfTime60-89DaysPastDueNotWorse": ("Payments 60 to 89 days late", "Times in the last two years.", "int",
        "times"),
    "NumberOfTimes90DaysLate": ("Payments 90 or more days late", "Times in the last two years.", "int", "times"),
}
GROUPS = [
    ("Borrower", ["age", "MonthlyIncome", "NumberOfDependents"]),
    ("Credit usage", ["RevolvingUtilizationOfUnsecuredLines", "DebtRatio", "NumberOfOpenCreditLinesAndLoans",
        "NumberRealEstateLoansOrLines"]),
    ("Payment history", ["NumberOfTime30-59DaysPastDueNotWorse", "NumberOfTime60-89DaysPastDueNotWorse",
        "NumberOfTimes90DaysLate"]),
]
OPTIONAL = {"MonthlyIncome", "NumberOfDependents"}
SENSITIVE = {"age"}


def label(c):
    return SPEC[c][0]


def build_groups():
    out = []
    for title, keys in GROUPS:
        fields = []
        for k in keys:
            lab, hint, kind, suffix = SPEC[k]
            st = META["ui_stats"][k]
            med = st["median"]
            if kind == "percent":
                default, step = f"{med * 100:.0f}", "any"
            elif kind == "money":
                default, step = f"{med:.0f}", "any"
            else:
                default, step = f"{med:.0f}", "1"
            fields.append({"key": k, "label": lab, "hint": hint, "kind": kind, "suffix": suffix,
                           "default": default, "step": step, "min": "18" if k == "age" else "0",
                           "unknown": k in OPTIONAL})
        out.append({"title": title, "fields": fields})
    return out

def explain(df):
```

```
    """Swap each input for the typical (median) value, all in one batch, and measure the change in risk."""
    cols = META["raw_cols"]
    frames = [df]
    for c in cols:
        alt = df.copy()
        alt[c] = META["ui_stats"][c]["median"]
        frames.append(alt)
    big = pd.concat(frames, ignore_index=True)
    probs = PIPE.predict_proba(add_features(big, META["caps"]))[:, 1]
    return float(probs[0]), {c: float(probs[0] - probs[i + 1]) for i, c in enumerate(cols)}


def fairness_view():
    w = {r["Age group"]: r for r in META["fair_with"]}
    wo = {r["Age group"]: r for r in META["fair_without"]}
    order = ["under 30", "30 to 59", "60 and over"]
    rows = [{"group": g, "n": w[g]["n"], "actual": w[g]["Actual default rate"], "pred": w[g]["Mean predicted risk"],
             "dec_with": w[g]["Declined"], "dec_without": wo[g]["Declined"],
             "fd_with": w[g]["Declined among non-defaulters"], "fd_without": wo[g]["Declined among non-defaulters"]}
            for g in order]
    young, old = rows[0], rows[2]
    ratio = young["fd_with"] / old["fd_with"] if old["fd_with"] else None
    return rows, {"ratio": ratio, "young_fd": young["fd_with"], "old_fd": old["fd_with"],
                  "auc_drop": META["auc_with_age"] - META["auc_without_age"]}


@app.template_filter("pct")
def pct(v, digits=1):
    return f"{v * 100:.{digits}f}%"


@app.template_filter("pct0")
def pct0(v):
    return f"{v * 100:.0f}%"


@app.get("/")
def index():
    fair_rows, fair_note = fairness_view()
    t_star, t_review = META["t_star"], META["t_review"]
    cost_gain = 1 - REPORTED_COST["model"] / min(REPORTED_COST["approve_all"], REPORTED_COST["decline_all"])
    imp = sorted(META["importance"].items(), key=lambda kv: -kv[1])
    imp_max = max(v for _, v in imp) or 1.0
    config = {"t_star": float(t_star), "t_review": float(t_review), "base_rate": float(META["base_rate"]),
              "axis_max": float(max(0.4, round(t_star * 2.5, 2)))}
    return render_template(
        "index.html", groups=build_groups(), config=config, meta=META, fair_rows=fair_rows, fair_note=fair_note,
        importance=[(label(k), (0.0 if abs(v) < 0.0005 else v), max(v, 0.0) / imp_max) for k, v in imp],
            cost=REPORTED_COST, cost_gain=cost_gain,
        t_star=t_star, t_review=t_review)


@app.post("/api/predict")
def predict():
    data = request.get_json(silent=True) or {}
    row = {}
    for c in META["raw_cols"]:
        v = data.get(c)
        if v is None:
            if c in OPTIONAL:
                row[c] = np.nan
                continue
            return jsonify(error=f"Please enter a value for {label(c)}."), 400
        try:
            x = float(v)
        except (TypeError, ValueError):
            return jsonify(error=f"{label(c)} must be a number."), 400
        if not math.isfinite(x) or x < 0 or x > 1e7:
            return jsonify(error=f"{label(c)} is outside the supported range."), 400
        row[c] = x
    if not 18 <= row["age"] <= 110:
        return jsonify(error="Age must be between 18 and 110."), 400

    df = pd.DataFrame([row])[META["raw_cols"]]
    p, effects = explain(df)
    t_star, t_review = META["t_star"], META["t_review"]
    band = "decline" if p >= t_star else "review" if p >= t_review else "approve"

    def items(pairs):
        return [{"label": label(c), "points": v * 100, "sensitive": c in SENSITIVE} for c, v in pairs]

    raising = sorted([kv for kv in effects.items() if kv[1] > 0.002], key=lambda kv: -kv[1])[:4]
    lowering = sorted([kv for kv in effects.items() if kv[1] < -0.002], key=lambda kv: kv[1])[:4]
    return jsonify(probability=p, band=band, base_rate=META["base_rate"],
                   raising=items(raising), lowering=items(lowering))


@app.get("/healthz")
def healthz():
    return "ok"


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 5000)), debug=False)
```

7. Output Snapshots

The snapshots below are the printed outputs of the executed Colab notebook (the final summary cell), shown as they were produced.

Snapshot 1. Dataset after cleaning and train/test split

```
Dataset: Give Me Some Credit, 149999 rows after cleaning, delinquency rate 0.0668
Train/test: 119999/30000
```

Snapshot 2. Cross-validated model comparison

```
CV results (5-fold, train set):
                              ROC-AUC             PR-AUC              Brier
Model
Logistic Regression     0.8560 +/- 0.0069  0.3848 +/- 0.0092  0.0500 +/- 0.0004
Random Forest           0.8631 +/- 0.0061  0.3992 +/- 0.0083  0.0492 +/- 0.0004
Hist Gradient Boosting  0.8622 +/- 0.0056  0.3974 +/- 0.0079  0.0492 +/- 0.0004
```

Snapshot 3. Selected model, tuning result and test metrics

```
Selected: Random Forest | params: {'model__max_depth': 12, 'model__min_samples_leaf': 20, 'select__k': 'all'} | CV AUC
    5-fold 0.8631, tuned 3-fold 0.8639
Test metrics: {'ROC-AUC': 0.8681, 'PR-AUC': 0.3956, 'Brier': 0.049} | base rate 0.0668
```

Snapshot 4. Cost-based decision rule on the test set

```
Decline threshold 0.160 (review from 0.080); cost per applicant: approve-all 0.3342, decline-all 0.9332, model 0.2107
Policy on test: {'Decline rate': 0.11, 'Default rate among approved': 0.031, 'Default rate among declined': 0.354, 'Share
    of all defaulters caught (recall)': 0.582, 'Precision of declines': 0.354, 'F1 of declines': 0.44}
```

Snapshot 5. Permutation importance of the inputs

```
Importance (drop in ROC-AUC):
RevolvingUtilizationOfUnsecuredLines    0.0549
NumberOfTime30-59DaysPastDueNotWorse    0.0441
NumberOfTimes90DaysLate                 0.0352
NumberOfTime60-89DaysPastDueNotWorse    0.0239
DebtRatio                               0.0114
age                                     0.0078
MonthlyIncome                           0.0050
NumberOfOpenCreditLinesAndLoans         0.0034
NumberOfDependents                      0.0016
NumberRealEstateLoansOrLines            0.0016
```

Snapshot 6. Fairness audit by age group

```
Age fairness: test AUC with age 0.8681, without 0.8641
WITH age:
              n  Actual default rate  Mean predicted risk  Declined  Declined among non-defaulters  Caught among defaulters
Age group
under 30   1803               0.126                0.113     0.196                          0.131                   0.643
30 to 59  18546               0.081                0.079     0.134                          0.094                   0.589
60 and over  9651               0.029                0.034     0.046                          0.033                   0.495
WITHOUT age:
              n  Actual default rate  Mean predicted risk  Declined  Declined among non-defaulters  Caught among defaulters
Age group
under 30   1803               0.126                0.104     0.190                          0.126                   0.634
30 to 59  18546               0.081                0.077     0.131                          0.092                   0.579
60 and over  9651               0.029                0.040     0.051                          0.038                   0.509
```

8. Results & Analysis

Model	ROC-AUC	PR-AUC	Brier (lower is better)
Logistic Regression	0.8560 +/- 0.0069	0.3848 +/- 0.0092	0.0500 +/- 0.0004
Random Forest (selected)	0.8631 +/- 0.0061	0.3992 +/- 0.0083	0.0492 +/- 0.0004
Hist Gradient Boosting	0.8622 +/- 0.0056	0.3974 +/- 0.0079	0.0492 +/- 0.0004

Table 1. 5-fold cross-validation on the training set (mean +/- standard deviation across folds).

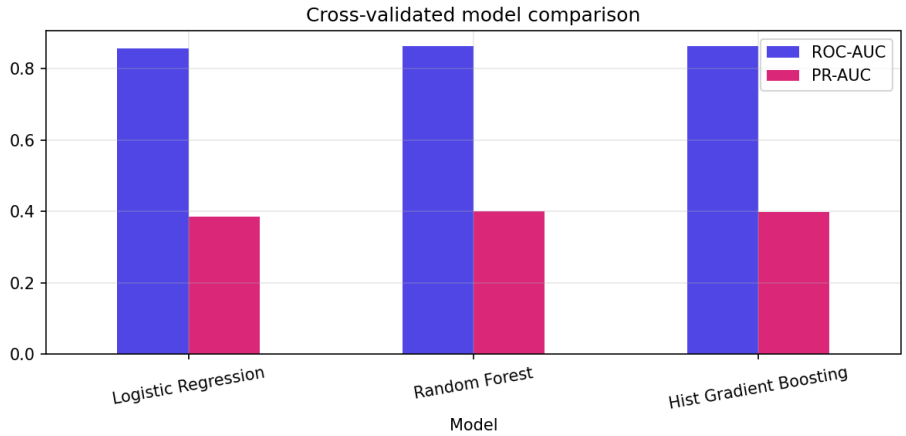


Figure 1. Cross-validated model comparison: mean ROC-AUC and PR-AUC over the 5 folds (output of the notebook).

Held-out test result (30,000 borrowers)	Value	Reference point
ROC-AUC	0.8681	0.5 = random ranking
PR-AUC	0.3956	0.0668 for a model with no skill (the default rate)
Brier score	0.0490	0.0623 when everyone is given the average risk

Table 2. Test-set ranking quality of the tuned Random Forest.

Decision rule at the 16% decline threshold	Value
Borrowers declined	11.0%
Default rate among approved borrowers	3.1% (overall rate 6.7%)
Default rate among declined borrowers	35.4%
Share of all defaulters caught (recall)	58.2%
Precision of declines / F1 of declines	35.4% / 0.44
Average cost per applicant: approve everyone / decline everyone / this model	0.334 / 0.933 / 0.211

Table 3. What the cost-based decision rule achieves on unseen borrowers.

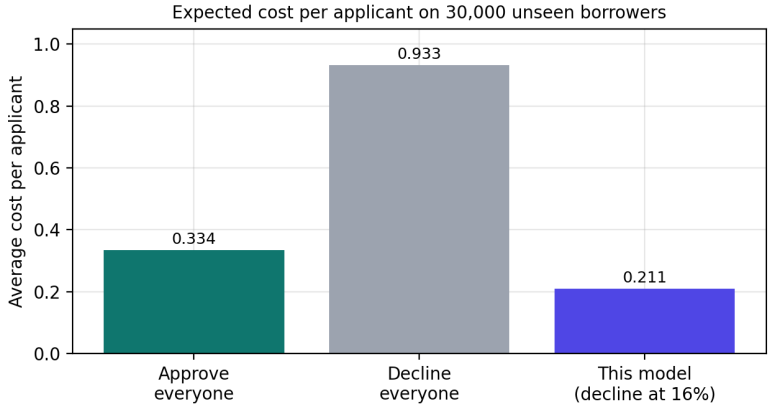


Figure 2. Expected cost per applicant under three policies (assumed 5:1 cost ratio; plotted from the notebook's reported values).

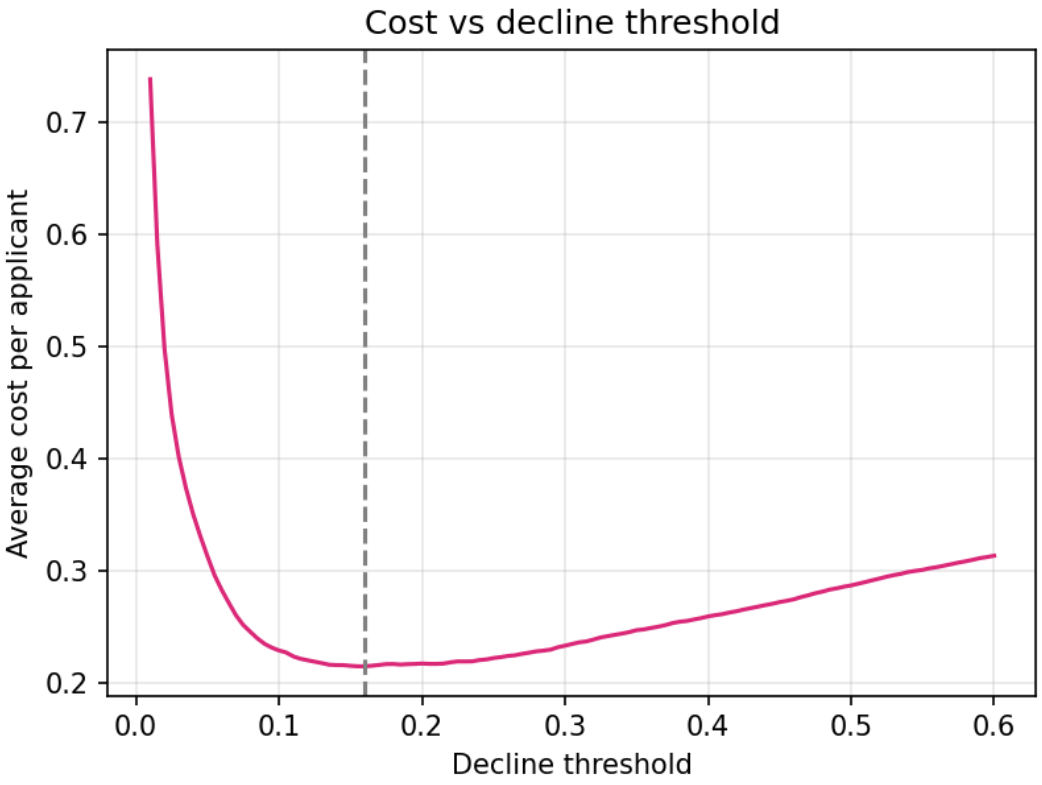


Figure 3. Average cost per applicant against the decline threshold, computed from cross-validated predictions on the training set. The dashed line marks the chosen 16% threshold (output of the notebook).

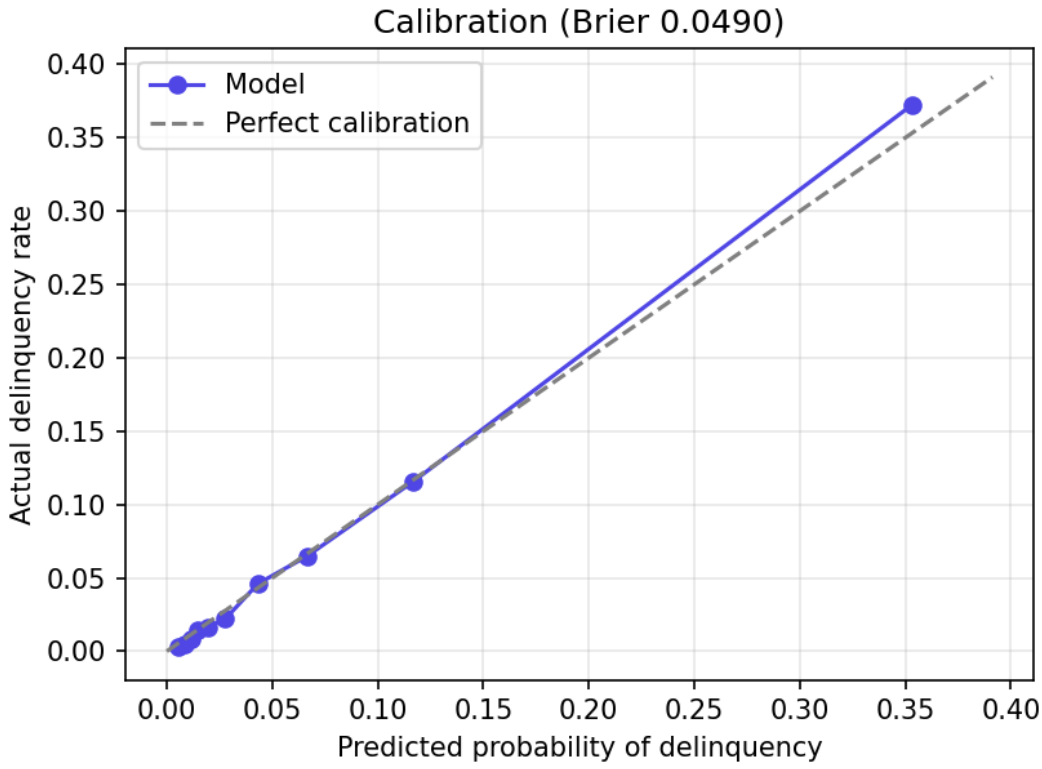


Figure 4. Calibration curve on the test set: predicted probability of delinquency against the actual delinquency rate in ten equal-sized risk groups (output of the notebook).

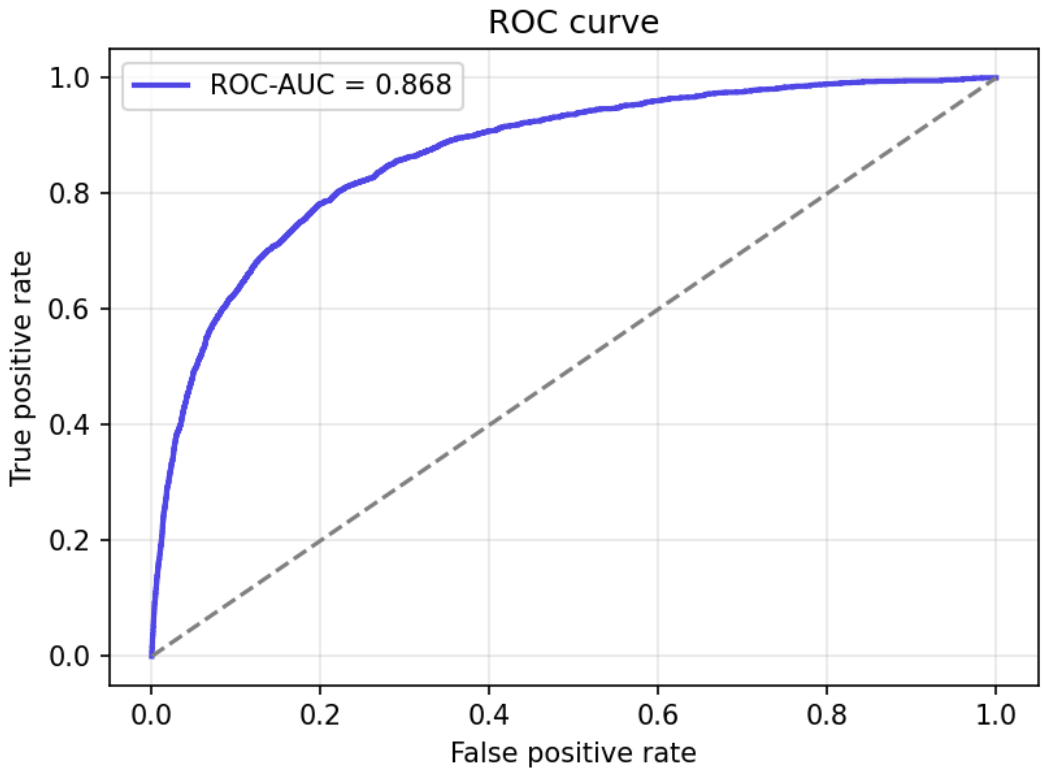


Figure 5. ROC curve of the tuned Random Forest on the test set, ROC-AUC 0.868 (output of the notebook).

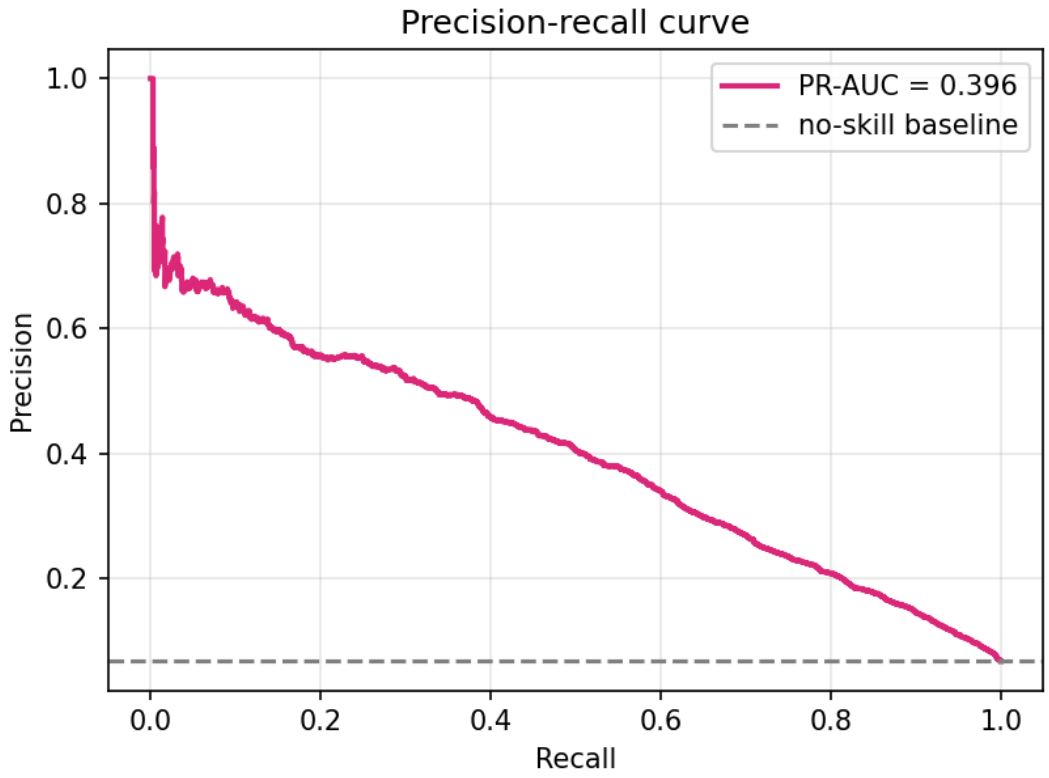


Figure 6. Precision-recall curve on the test set, PR-AUC 0.396. The dashed line is the no-skill baseline, equal to the default rate of 0.067 (output of the notebook).

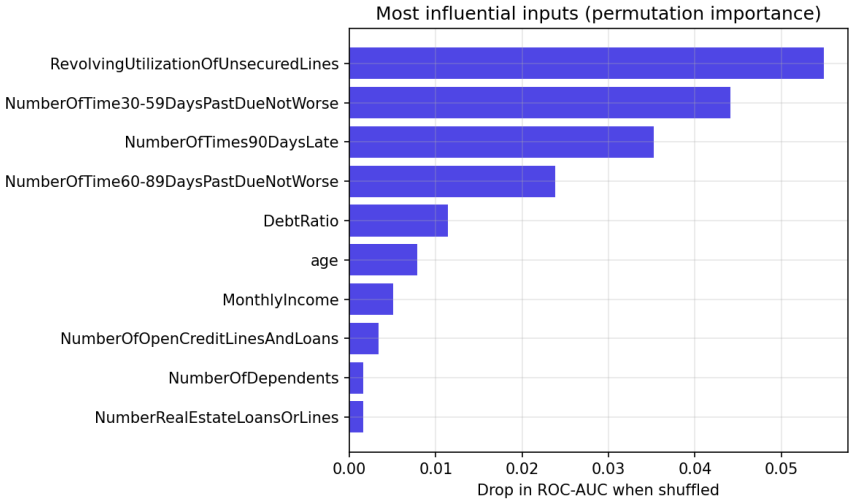


Figure 7. Permutation importance of the original inputs on the test set (output of the notebook).

Analysis

- **Accuracy is the wrong yardstick here.** Only 6.7% of borrowers default, so approving everyone is already 93% accurate while catching no defaulter. ROC-AUC, PR-AUC, Brier score and the cost of decisions measure what matters.
- **Model comparison.** Random Forest (0.8631) and Hist Gradient Boosting (0.8622) differ by 0.0009, far less than the fold-to-fold spread of about 0.006, so they are effectively tied. Logistic Regression is about 0.007 lower, roughly one standard deviation. Tuning moved the cross-validated score from 0.8631 to 0.8639 (measured on different folds), which is not a meaningful change.
- **Generalization.** The test ROC-AUC of 0.8681 sits within about one standard deviation of the cross-validated 0.8631, so there is no sign of overfitting or leakage. The PR-AUC of 0.3956 is about six times the no-skill level of 0.0668, and the Brier score of 0.0490 is about 21% below the 0.0623 of a model that gives everyone the average risk.
- **What the decision rule does.** Declining the 11% of borrowers with the highest predicted risk catches 58% of all defaulters. Among approved borrowers the default rate falls to 3.1% from 6.7%. Under the assumed 5:1 cost ratio the expected cost per applicant is 0.211, which is 37% below the best policy without a model (approve everyone, 0.334). The threshold came from cross-validation on training data and was checked on test data the model had never seen. The cost ratio itself is an assumption, and a real lender would use its own figures.
- **What drives the estimate.** Credit utilization and the three late-payment counts dominate. The late-payment columns are correlated, so their individual importances understate how much past-due history matters as a group. Age, income and debt ratio contribute modestly.

Fairness audit (age)

Age group	n	Actual default	Mean predicted risk	Declined (with age)	Declined (no age)	Good borrowers declined (with)	Good borrowers declined (no age)	Defaulters caught (with)	Defaulters caught (no age)
under 30	1,803	12.6%	11.3%	19.6%	19.0%	13.1%	12.6%	64.3%	63.4%
30 to 59	18,546	8.1%	7.9%	13.4%	13.1%	9.4%	9.2%	58.9%	57.9%
60 and over	9,651	2.9%	3.4%	4.6%	5.1%	3.3%	3.8%	49.5%	50.9%

Table 4. Decisions by age group. Test ROC-AUC is 0.8681 with age and 0.8641 without age.

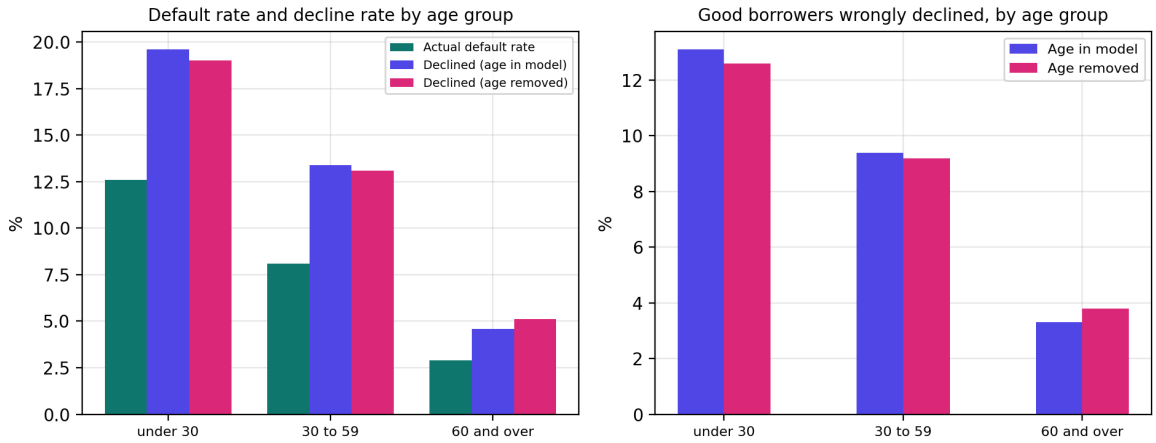


Figure 8. Default and decline rates by age group, with and without age in the model (plotted from the notebook's reported values).

- **Predictions track reality on average in every group.** Mean predicted risk (11.3%, 7.9%, 3.4%) is close to the actual default rate (12.6%, 8.1%, 2.9%), so the model is not simply over-penalizing one group on average.
- **The burden of errors is uneven.** Declines range from 19.6% of borrowers under 30 to 4.6% of those aged 60 and over, which partly reflects real differences in default rates. But among borrowers who did not default, 13.1% of those under 30 were still declined versus 3.3% of those aged 60 and over, about four times as often.
- **Removing age does not fix it.** Without age the ROC-AUC drops by only 0.004 and decline rates barely move (19.0%, 13.1%, 5.1%). The likely reason is that other inputs, such as credit history and utilization, carry age-related signal and act as proxies. This explanation was not tested directly.
- **Caution.** The under-30 group has 1,803 borrowers, so its rates carry roughly one point of sampling noise. Which fairness criterion is acceptable is a policy and legal decision, and this audit is not a compliance review.

Limitations

- The data is anonymized US consumer data from around 2011, so the model cannot be assumed to transfer to other countries or time periods.
- The target is serious delinquency, not profit, and the 5:1 cost ratio is an assumption.
- Final numbers come from one train/test split, although cross-validation agrees closely with it.

9. Conclusion

The project delivered an end-to-end, explainable credit-risk decision-support system trained on 150,000 real borrowers. The tuned Random Forest reaches a test ROC-AUC of 0.868 and well-behaved probabilities. A cost-based rule declines 11% of applicants, catches 58% of eventual defaulters and lowers the expected cost per applicant by 37% against the best policy without a model, under the stated assumption, while the default rate among approved borrowers falls to 3.1% from 6.7%. The system explains its estimates and was audited by age, which showed that younger good borrowers are declined more often and that removing age alone does not remove the difference. It is provided as an interactive interface in the notebook and as a web application.

Future work. Recalibrate the probabilities (for example with isotonic regression), add SHAP explanations, run a sensitivity analysis over several cost ratios, test fairness-aware thresholds, validate on data from a later period, and monitor for drift after deployment.